



Project File

Web Programming using Python and JavaScript

Submitted By:

Name: Aditya Pal

Course: B.Tech CSE

Year: 2nd Year, 4th Semester

Roll No: 2401010119

Submitted To:

Name: Dr. Mansi Kajal

EXPERIMENT – 1

❖ **AIM:** Personal Portfolio Website using HTML, CSS Styling and JavaScript.

❖ **Tasks:**

- **Project Setup & Introduction** – Create Project folder and Basic linked files.
- **HTML Structure** – Build portfolio section using semantic HTML.
- **Styling with CSS** – Apply layout, colors, spacing, and neat design.
- **Projects & Skills Display** – Show skills and projects clearly on the page.
- **JavaScript Interactivity** – Add basic interactive functionality using JavaScript.
- **Contact Form** – Create a form with user submission feedback.

❖ **Tech Stack Used:**

- **HTML, CSS, and JavaScript**

❖ **CODES:**

➤ **HTML Code:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Aditya Solutions</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
```

```

<header class="hero">
  <h1>Aditya Solutions</h1>
  <p>Empowering Businesses Through Digital Innovation</p>
  <a href="#contact" class="btn">Get Started</a>
</header>

<section id="about">
  <h2>About Us</h2>
  <p>
    Aditya Solutions is a forward-thinking technology company
delivering
    innovative digital solutions that help businesses grow, scale,
    and succeed in the modern digital world.
  </p>
</section>

<section id="services">
  <h2>Our Services</h2>

  <div class="services-container">
    <div class="service-card">
      <h3>Web Development</h3>
      <p>Modern, responsive, and high-performing websites
tailored for your brand.</p>
    </div>

    <div class="service-card">
      <h3>Mobile App Development</h3>
      <p>Powerful Android and iOS applications designed for
seamless user experience.</p>
    </div>

    <div class="service-card">
      <h3>Digital Marketing</h3>
      <p>Boost your visibility with strategic SEO and targeted
marketing campaigns.</p>
    </div>
  </div>
</section>

<section id="testimonials">
  <h2>What Our Clients Say</h2>

  <div class="testimonial">
    <p>"Aditya Solutions transformed our online presence
completely."</p>
    <span>- Sarah Williams</span>
  </div>

```

```

        <div class="testimonial">
            <p>Exceptional service, creative ideas, and on-time
delivery.</p>
            <span>- James Anderson</span>
        </div>
    </section>

    <section id="contact">
        <h2>Contact Us</h2>

        <div class="contact-info">
            <p>Email: info@adityasolutions.com</p>
            <p>Phone: +1 234 567 890</p>
            <p>Address: 123 Innovation Street, Silicon City</p>
        </div>

        <form class="contact-form">
            <input type="text" placeholder="Your Name" required>
            <input type="email" placeholder="Your Email" required>
            <textarea placeholder="Your Message" rows="4"
required></textarea>
            <button type="submit">Send Message</button>
        </form>
    </section>

    <footer>
        <p>&copy; 2026 Aditya Solutions. All rights reserved.</p>
    </footer>

</body>
</html>

```

➤ CSS Code:

```

body {
    font-family: 'Segoe UI', sans-serif;
    margin: 0;
    padding: 0;
    background: linear-gradient(to right, #f8fafc, #e2e8f0);
    color: #333;
    scroll-behavior: smooth;
}

.hero {
    background: linear-gradient(135deg, #4f46e5, #06b6d4);
    color: white;

```

```

        text-align: center;
        padding: 100px 20px;
    }

    .hero h1 {
        font-size: 48px;
        margin-bottom: 15px;
    }

    .hero p {
        font-size: 20px;
        margin-bottom: 25px;
    }

    .btn {
        display: inline-block;
        padding: 12px 25px;
        background-color: white;
        color: #4f46e5;
        font-weight: bold;
        text-decoration: none;
        border-radius: 30px;
        transition: 0.3s ease;
    }

    .btn:hover {
        background-color: #e0e7ff;
        transform: translateY(-3px);
    }

    section {
        padding: 60px 30px;
        margin: 40px auto;
        max-width: 1100px;
        background-color: white;
        border-radius: 12px;
        box-shadow: 0 10px 25px rgba(0, 0, 0, 0.05);
    }

    section h2 {
        text-align: center;
        margin-bottom: 30px;
        font-size: 28px;
        color: #4f46e5;
    }

    .services-container {
        display: flex;

```

```

    gap: 25px;
    flex-wrap: wrap;
    justify-content: center;
}

.service-card {
    flex: 1;
    min-width: 260px;
    padding: 25px;
    border-radius: 10px;
    background: #f9fafb;
    transition: 0.3s ease;
    border: 1px solid #e5e7eb;
}

.service-card:hover {
    transform: translateY(-8px);
    box-shadow: 0 15px 25px rgba(0, 0, 0, 0.08);
}

.testimonial {
    padding: 20px;
    margin-bottom: 20px;
    background: #f1f5f9;
    border-left: 5px solid #4f46e5;
    border-radius: 6px;
}

.testimonial span {
    display: block;
    margin-top: 10px;
    font-weight: bold;
    color: #4f46e5;
}

.contact-info {
    text-align: center;
    margin-bottom: 25px;
}

.contact-form input,
.contact-form textarea {
    width: 100%;
    padding: 12px;
    margin-bottom: 15px;
    border-radius: 6px;
    border: 1px solid #d1d5db;
    font-size: 14px;
}

```

```
}

.contact-form input:focus,
.contact-form textarea:focus {
  outline: none;
  border-color: #4f46e5;
  box-shadow: 0 0 0 2px rgba(79, 70, 229, 0.2);
}

.contact-form button {
  background: linear-gradient(135deg, #4f46e5, #06b6d4);
  color: white;
  border: none;
  padding: 12px 25px;
  border-radius: 25px;
  cursor: pointer;
  font-size: 15px;
  transition: 0.3s ease;
}

.contact-form button:hover {
  opacity: 0.9;
  transform: translateY(-2px);
}

footer {
  text-align: center;
  padding: 20px;
  background: #1e293b;
  color: white;
  margin-top: 40px;
}
```

❖ OUTPUT:

Aditya Solutions

Empowering Businesses Through Digital Innovation

[Get Started](#)

About Us

Aditya Solutions is a forward-thinking technology company delivering innovative digital solutions that help businesses grow, scale, and succeed in the modern digital world.

Our Services

Web Development

Modern, responsive, and high-performing websites tailored for your brand.

Mobile App Development

Powerful Android and iOS applications designed for seamless user experience.

Digital Marketing

Boost your visibility with strategic SEO and targeted marketing campaigns.

What Our Clients Say

"Aditya Solutions transformed our online presence completely."

- Sarah Williams

"Exceptional service, creative ideas, and on-time delivery."

- James Anderson

Contact Us

Email: info@adityasolutions.com

Phone: +1 234 567 890

Address: 123 Innovation Street, Silicon City

Contact Us

Email: info@adityasolutions.com

Phone: +1 234 567 890

Address: 123 Innovation Street, Silicon City

Send Message

© 2026 Aditya Solutions. All rights reserved.

EXPERIMENT – 2

❖ **AIM: Build a Company Landing Page using HTML and CSS.**

❖ **Tasks:**

- **Project Setup & Introduction** – Create a project folder with required files index.html and style.css
- **HTML Structure** – Building HTML structure including Hero Section, About section, Services, Testimonials, Contact Information.
- **Styling with CSS** – Apply CSS for Page layout, colors/fonts, alignment, margin/padding.
- **Service Section Design** – Adding atleast 3 services, including service name, short description
- **Testimonials** – Adding atleast 2 customer testimonials including customer name, and their feedback.
- **Contact Form** – Adding a contact form with address/email/phone

❖ Codes:

➤ HTML Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Aditya Pal | Portfolio</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <header class="hero">
    <div class="hero-content">
      <h1>Aditya Pal</h1>
      <p>B.Tech CSE Student</p>
      <p>K.R. Mangalam University | Second Year</p>
      <a href="#contact" class="btn">Contact Me</a>
    </div>
  </header>

  <section id="about" class="card">
    <h2>About Me</h2>
    <p>
      I am Aditya Pal, a second-year B.Tech Computer Science
      Engineering student
      at K.R. Mangalam University. I am passionate about programming
      and continuously
      learning new technologies to enhance my skills and build
      impactful projects.
    </p>
  </section>

  <section id="skills" class="card">
    <h2>Skills</h2>
    <ul class="skills-list">
      <li>HTML</li>
      <li>CSS</li>
      <li>Python</li>
      <li>Java</li>
      <li>C++</li>
    </ul>
  </section>

  <section id="education" class="card">
    <h2>Education</h2>
```

```

        <p>
            <strong>Bachelor of Technology (B.Tech)</strong><br>
            Computer Science Engineering<br>
            K.R. Mangalam University<br>
            Currently in Second Year
        </p>
    </section>

    <section id="contact" class="card">
        <h2>Contact</h2>
        <p>Email: adityapal@example.com</p>
        <p>Phone: +91 9999999999</p>
    </section>

    <footer>
        <p>&copy; 2026 Aditya Pal. All rights reserved.</p>
    </footer>

</body>
</html>

```

➤ CSS Code:

```

body {
    margin: 0;
    padding: 0;
    font-family: 'Segoe UI', sans-serif;
    background: linear-gradient(135deg, #667eea, #764ba2);
    color: #333;
}

.hero {
    height: 90vh;
    display: flex;
    justify-content: center;
    align-items: center;
    text-align: center;
    color: white;
    background: linear-gradient(135deg, #1e3c72, #2a5298);
}

.hero-content h1 {
    font-size: 48px;
    margin-bottom: 10px;
}

.hero-content p {

```

```

    font-size: 18px;
    margin: 5px 0;
}

.btn {
    display: inline-block;
    margin-top: 20px;
    padding: 12px 25px;
    background: white;
    color: #2a5298;
    border-radius: 30px;
    text-decoration: none;
    font-weight: bold;
    transition: 0.3s ease;
}

.btn:hover {
    background: #e0e7ff;
    transform: translateY(-3px);
}

.card {
    max-width: 900px;
    margin: 40px auto;
    padding: 35px;
    border-radius: 15px;
    background: rgba(255, 255, 255, 0.95);
    box-shadow: 0 15px 35px rgba(0, 0, 0, 0.15);
}

.card h2 {
    text-align: center;
    margin-bottom: 20px;
    color: #2a5298;
}

.skills-list {
    list-style: none;
    padding: 0;
    display: flex;
    gap: 15px;
    flex-wrap: wrap;
    justify-content: center;
}

.skills-list li {
    background: linear-gradient(135deg, #667eea, #764ba2);
    color: white;

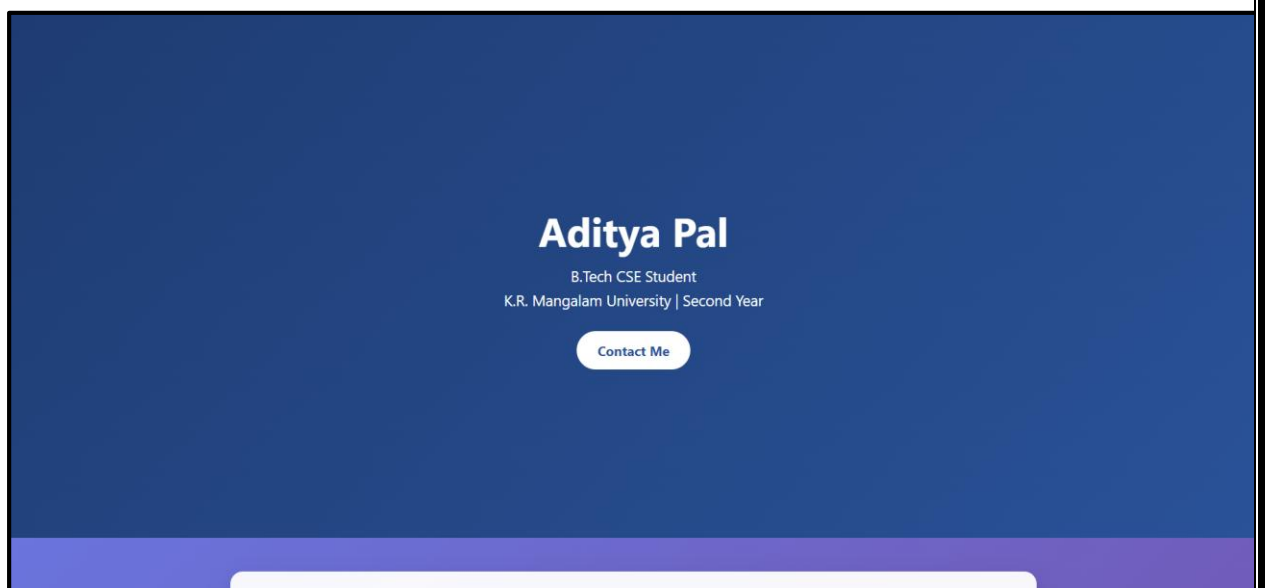
```

```
padding: 10px 18px;
border-radius: 25px;
font-weight: bold;
transition: 0.3s ease;
}

.skills-list li:hover {
  transform: scale(1.1);
  cursor: pointer;
}

footer {
  text-align: center;
  padding: 20px;
  background: #1e3c72;
  color: white;
  margin-top: 40px;
}
```

❖ OUTPUT:



About Me

I am Aditya Pal, a second-year B.Tech Computer Science Engineering student at K.R. Mangalam University. I am passionate about programming and continuously learning new technologies to enhance my skills and build impactful projects.

Skills

HTMLCSSPythonJavaC++

Education

Bachelor of Technology (B.Tech)
Computer Science Engineering
K.R. Mangalam University
Currently in Second Year

Education

Bachelor of Technology (B.Tech)
Computer Science Engineering
K.R. Mangalam University
Currently in Second Year

Contact

Email: adityapal@example.com
Phone: +91 9999999999

© 2026 Aditya Pal. All rights reserved.

EXPERIMENT-3

❖ **AIM: Build an Interactive Quiz Application using HTML, CSS, and JavaScript.**

❖ **Tasks:**

- **Project Setup & Introduction** – Create a project folder named interactive_quiz with required files: index.html, style.css, and script.js. Add header comments in HTML (project title, name, date) and display a brief quiz introduction.
- **HTML Structure** – Build a structured quiz interface including question text, multiple-choice options (radio buttons or buttons), and a “Next” or “Submit” button using semantic HTML elements.
- **Styling with CSS** – Apply CSS to improve readability, align buttons and sections properly, and use appropriate colors and spacing for better user experience.

- **JavaScript Logic & Interactivity** – Store quiz questions and answers in JavaScript (array/object), capture user-selected answers, navigate between questions using button events, and track/update the score dynamically.
- **Result Display** – Show final score after quiz completion along with a feedback message such as “Excellent”, “Good”, or “Try Again”. Optionally hide the quiz interface after submission.
- **Restart Quiz Feature (Bonus)** – Add a “Restart Quiz” button to reset score and question index, making the application reusable.

❖ Codes:

➤ HTML Code:

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Interactive Quiz | Aditya Pal</title>

  <!-- Google Font -->
  <link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;500;600;700&display=swap" rel="stylesheet">

  <link rel="stylesheet" href="style.css">
</head>

<body>

  <main class="quiz-container">
```

```

<section id="intro">
  <h1>Interactive Quiz</h1>
  <p>Test your programming and web development basics</p>
</section>

<div class="progress">
  <div id="progress-bar"></div>
</div>

<section id="quiz">
  <h2 id="question"></h2>

  <div class="options">
    <label class="option-item">
      <input type="radio" name="option" value="0">
      <span id="opt0"></span>
    </label>

    <label class="option-item">
      <input type="radio" name="option" value="1">
      <span id="opt1"></span>
    </label>

    <label class="option-item">
      <input type="radio" name="option" value="2">
      <span id="opt2"></span>
    </label>

    <label class="option-item">
      <input type="radio" name="option" value="3">
      <span id="opt3"></span>
    </label>
  </div>

  <button id="nextBtn">Next Question</button>
</section>

<section id="result" class="hidden">
  <h2>Quiz Result</h2>
  <p id="scoreText"></p>
  <p id="feedback"></p>

  <div id="review"></div>

  <button id="restartBtn">Restart Quiz</button>
</section>

</main>

```

```
<script src="script.js"></script>
</body>
</html>
```

➤ CSS Code:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: "Poppins", "Segoe UI", sans-serif;
}

body {
  min-height: 100vh;
  background: linear-gradient(135deg, #667eea, #764ba2, #6a11cb);
  background-size: 300% 300%;
  animation: gradientMove 8s ease infinite;
  display: flex;
  justify-content: center;
  align-items: center;
}

@keyframes gradientMove {
  0% { background-position: 0% 50%; }
  50% { background-position: 100% 50%; }
  100% { background-position: 0% 50%; }
}

.quiz-container {
  width: 520px;
  max-width: 95%;
  padding: 35px;
  border-radius: 20px;
  background: rgba(255, 255, 255, 0.12);
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
  border: 1px solid rgba(255, 255, 255, 0.25);
  box-shadow: 0 25px 50px rgba(0, 0, 0, 0.25);
  animation: fadeIn 0.6s ease-in-out;
  color: white;
}

@keyframes fadeIn {
  from {
```

```

        opacity: 0;
        transform: translateY(25px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

#intro {
    text-align: center;
    margin-bottom: 25px;
}

#intro h1 {
    font-size: 30px;
    margin-bottom: 8px;
    letter-spacing: 1px;
}

#intro p {
    font-size: 14px;
    opacity: 0.85;
}

.progress {
    width: 100%;
    height: 10px;
    background: rgba(255, 255, 255, 0.2);
    border-radius: 10px;
    overflow: hidden;
    margin-bottom: 25px;
}

#progress-bar {
    height: 100%;
    width: 0%;
    background: linear-gradient(90deg, #00f5a0, #00d9f5);
    box-shadow: 0 0 15px rgba(0, 255, 200, 0.8);
    transition: width 0.4s ease;
}

#question {
    font-size: 18px;
    margin-bottom: 18px;
    font-weight: 500;
}

```

```

.options label {
  display: flex;
  align-items: center;
  padding: 14px;
  margin-bottom: 12px;
  border-radius: 12px;
  background: rgba(255, 255, 255, 0.15);
  cursor: pointer;
  transition: all 0.3s ease;
  border: 1px solid transparent;
}

.options label:hover {
  background: rgba(255, 255, 255, 0.25);
  transform: translateY(-2px);
  border: 1px solid rgba(255, 255, 255, 0.4);
}

.options input {
  margin-right: 10px;
  accent-color: #00f5a0;
  cursor: pointer;
}

button {
  width: 100%;
  padding: 14px;
  border-radius: 12px;
  border: none;
  background: linear-gradient(135deg, #00f5a0, #00d9f5);
  color: #111;
  font-size: 16px;
  font-weight: 600;
  cursor: pointer;
  margin-top: 18px;
  transition: all 0.3s ease;
}

button:hover {
  transform: translateY(-2px);
  box-shadow: 0 8px 20px rgba(0, 255, 200, 0.5);
}

.hidden {
  display: none;
}

#result {

```

```
        text-align: center;
    }

    #result h2 {
        margin-bottom: 12px;
    }

    #scoreText {
        font-size: 20px;
        margin-bottom: 5px;
    }

    #feedback {
        font-weight: bold;
        font-size: 16px;
        margin-bottom: 20px;
        color: #00f5a0;
    }

    .review-item {
        text-align: left;
        background: rgba(255, 255, 255, 0.15);
        padding: 14px;
        border-radius: 12px;
        margin-bottom: 12px;
        border-left: 4px solid #00f5a0;
    }

    .review-item p {
        font-size: 14px;
        margin-bottom: 5px;
    }

    .correct {
        color: #00ff99;
        font-weight: 600;
    }

    .wrong {
        color: #ff6b6b;
        font-weight: 600;
    }

    @media (max-width: 500px) {
        .quiz-container {
            padding: 25px;
        }
    }
}
```

```
#intro h1 {
    font-size: 24px;
}

#question {
    font-size: 16px;
}
}
```

➤ JAVASCRIPT Code:

```
const quizData = [
  {
    question: "What does HTML stand for?",
    options: [
      "Hyper Text Markup Language",
      "High Text Machine Language",
      "Hyperlinks and Text Markup Language",
      "Home Tool Markup Language"
    ],
    answer: 0
  },
  {
    question: "Which language is used for styling web pages?",
    options: ["HTML", "Java", "CSS", "Python"],
    answer: 2
  },
  {
    question: "Which tag creates a hyperlink?",
    options: ["<link>", "<a>", "<href>", "<url>"],
    answer: 1
  },
  {
    question: "Which language runs in the browser?",
    options: ["Java", "C++", "Python", "JavaScript"],
    answer: 3
  },
  {
    question: "CSS comments use which syntax?",
    options: ["//", "#", "/* */", "<!-- -->"],
    answer: 2
  },
  {
    question: "Which keyword declares a variable in JS?",
    options: ["var", "int", "string", "define"],
    answer: 0
  },
],
```

```

    {
      question: "Which is a semantic HTML tag?",
      options: ["<div>", "<span>", "<section>", "<b>"],
      answer: 2
    },
    {
      question: "CSS property for text size?",
      options: ["font-style", "text-size", "font-size", "size"],
      answer: 2
    },
    {
      question: "Java was developed by?",
      options: ["Microsoft", "Sun Microsystems", "Google", "Apple"],
      answer: 1
    },
    {
      question: "Function keyword in JavaScript?",
      options: ["method", "function", "def", "func"],
      answer: 1
    }
  ]
];

let currentQuestion = 0;
let score = 0;
let userAnswers = [];

const questionEl = document.getElementById("question");
const options = document.querySelectorAll("input[name='option']");
const nextBtn = document.getElementById("nextBtn");
const quizSection = document.getElementById("quiz");
const resultSection = document.getElementById("result");
const scoreText = document.getElementById("scoreText");
const feedback = document.getElementById("feedback");
const restartBtn = document.getElementById("restartBtn");
const reviewDiv = document.getElementById("review");
const progressBar = document.getElementById("progress-bar");

function loadQuestion() {
  const q = quizData[currentQuestion];

  questionEl.textContent = `Q${currentQuestion + 1}. ${q.question}`;

  q.options.forEach((opt, i) => {
    document.getElementById("opt" + i).textContent = opt;
    options[i].checked = false;
  });
}

```



```

        progressBar.style.width = `${(currentQuestion / quizData.length) *
100}%`;

        nextBtn.textContent =
            currentQuestion === quizData.length - 1
                ? "Submit Quiz"
                : "Next Question";
    }

    function getSelected() {
        let selected = null;
        options.forEach(opt => {
            if (opt.checked) {
                selected = parseInt(opt.value);
            }
        });
        return selected;
    }

    nextBtn.addEventListener("click", () => {
        const selected = getSelected();

        if (selected === null) {
            alert("Please select an option before continuing.");
            return;
        }

        userAnswers.push(selected);

        if (selected === quizData[currentQuestion].answer) {
            score++;
        }

        currentQuestion++;

        if (currentQuestion < quizData.length) {
            loadQuestion();
        } else {
            showResult();
        }
    });

    function showResult() {
        quizSection.classList.add("hidden");
        resultSection.classList.remove("hidden");
        progressBar.style.width = "100%";

        scoreText.textContent = `Your Score: ${score} / ${quizData.length}`;
    }

```

```

    if (score >= 8) {
        feedback.textContent = "Outstanding Performance! 🏆";
    } else if (score >= 5) {
        feedback.textContent = "Great Job! Keep Improving 👍";
    } else {
        feedback.textContent = "Keep Practicing 💡";
    }

    reviewDiv.innerHTML = "";

    quizData.forEach((q, i) => {
        const div = document.createElement("div");
        div.classList.add("review-item");

        div.innerHTML = `
            <p><strong>Q${i + 1}. ${q.question}</strong></p>
            <p class="${userAnswers[i] === q.answer ? "correct" :
"wrong"}">
                Your Answer: ${q.options[userAnswers[i]]}
            </p>
            <p class="correct">
                Correct Answer: ${q.options[q.answer]}
            </p>
        `;

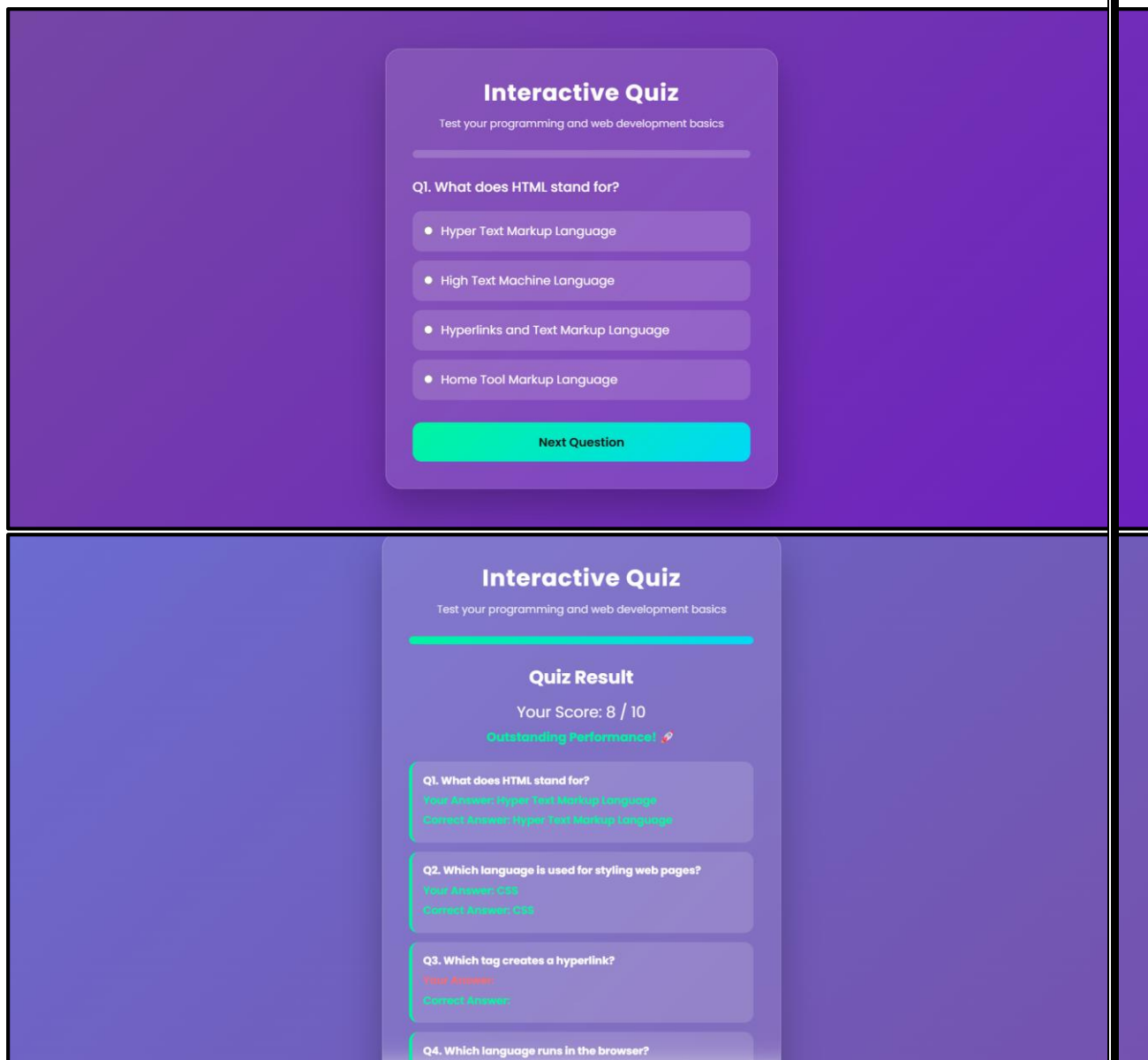
        reviewDiv.appendChild(div);
    });

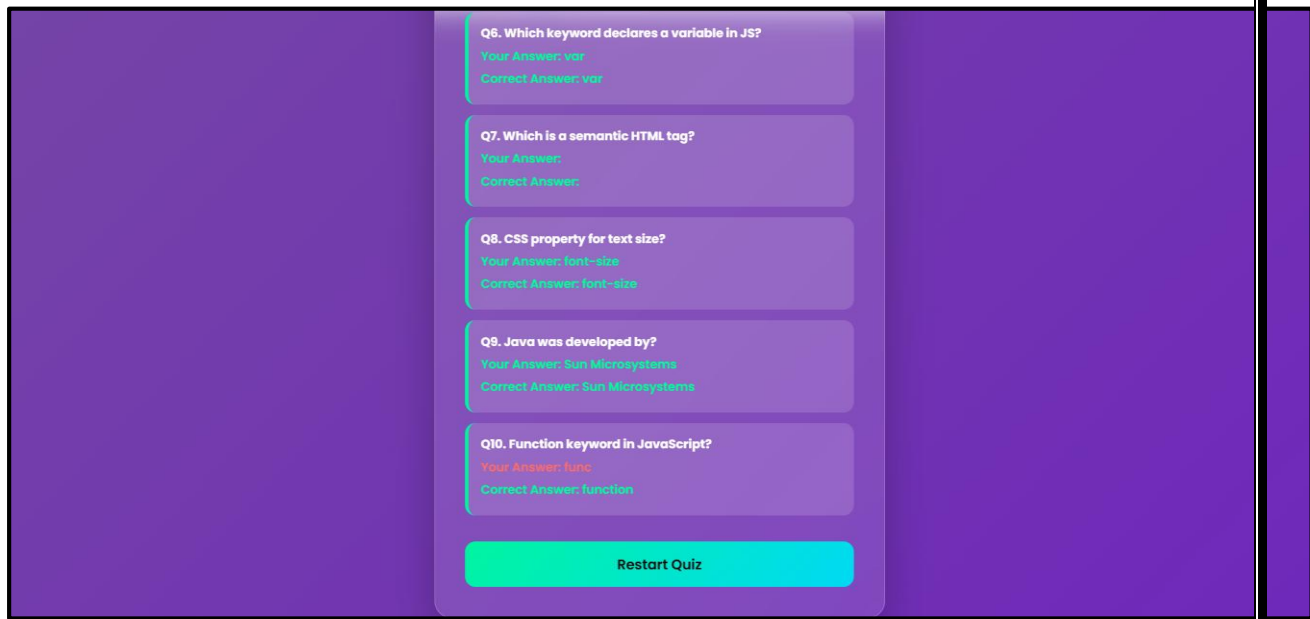
    restartBtn.addEventListener("click", () => {
        currentQuestion = 0;
        score = 0;
        userAnswers = [];
        resultSection.classList.add("hidden");
        quizSection.classList.remove("hidden");
        loadQuestion();
    });

    loadQuestion();

```

❖ OUTPUT:





EXPERIMENT-4

❖ **AIM: Build a Dynamic To-Do List Application using HTML, CSS, and JavaScript.**

❖ **Tasks:**

- **Project Setup & Introduction** – Create a project folder named `dynamic_todo_list` containing the required files: `index.html`, `style.css`, and `script.js`. Add header comments in the HTML file (project title, name, date) and display a short description of the to-do list application.
- **To-Do List Interface (HTML)** – Design the basic interface using HTML. Create an input field for entering tasks, an “Add Task” button, and display tasks in a list format using `<ul>` or `<ol>` elements.
- **Styling the To-Do List (CSS)** – Apply CSS styling to improve the interface appearance. Style the input box and buttons, align the task list properly, and visually separate tasks to keep the layout clean and readable.
- **Add & Display Tasks (JavaScript)** – Use JavaScript to capture user input and dynamically add new tasks to the list. Implement validation to prevent empty tasks from being added.
- **Edit & Delete Tasks** – Add “Edit” and “Delete” buttons for each task. Allow users to modify the task text and remove tasks dynamically from the list.
- **Task Completion Status (Bonus)** – Add a checkbox or toggle option to mark tasks as completed. Display completed tasks

with a visual indicator such as strikethrough text, and optionally show the count of completed vs pending tasks.

- **Submission Instructions:**

- Submit the dynamic_todo_list project folder.
- Include index.html, style.css, and script.js files.
- Provide at least 2 screenshots (before and after task updates).
- Ensure the code is clean and well-commented.
- Submit the GitHub repository URL via LMS.

❖ **Codes:**

➤ **HTML Code:**

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Smart To-Do List</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="app-container">

  <h1>🌟 To-Do List</h1>
  <p class="subtitle">Stay organized. Stay productive.</p>

  <div class="input-group">
    <input type="text" id="taskInput" placeholder="What needs to be done?">
    <button id="addTaskBtn">+</button>
  </div>

  <div class="stats">
    <span id="taskCount">0 Tasks</span>
    <span id="completedCount">0 Completed</span>
  </div>

</div>
```

```

<div class="progress-container">
  <div id="progressBar"></div>
</div>

<ul id="taskList"></ul>

<button id="clearAllBtn" class="clear-btn">Clear All</button>

</div>

<script src="script.js"></script>
</body>
</html>

```

➤ CSS Code:

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: "Segoe UI", Tahoma, sans-serif;
  background-color: #f4f6f8;
  display: flex;
  justify-content: center;
  padding: 40px 15px;
}

.app-container {
  width: 100%;
  max-width: 520px;
  background-color: #ffffff;
  padding: 30px;
  border-radius: 8px;
  box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
}

h1 {
  margin: 0;
  font-size: 24px;
  font-weight: 600;
  color: #1f2937;
}

```

```

        text-align: center;
    }

    .subtitle {
        text-align: center;
        font-size: 14px;
        color: #6b7280;
        margin: 8px 0 25px;
    }

    .input-group {
        display: flex;
    }

    #taskInput {
        flex: 1;
        padding: 10px 12px;
        border: 1px solid #d1d5db;
        border-radius: 6px 0 0 6px;
        font-size: 14px;
        outline: none;
    }

    #taskInput:focus {
        border-color: #2563eb;
    }

    #addTaskBtn {
        width: 50px;
        border: none;
        background-color: #2563eb;
        color: white;
        font-size: 18px;
        border-radius: 0 6px 6px 0;
        cursor: pointer;
        transition: background 0.2s ease;
    }

    #addTaskBtn:hover {
        background-color: #1e4fd8;
    }

    .stats {
        display: flex;
        justify-content: space-between;
        margin: 18px 0 8px;
    }

```



```
font-size: 13px;
color: #4b5563;
}

.progress-container {
  width: 100%;
  height: 6px;
  background-color: #e5e7eb;
  border-radius: 4px;
  margin-bottom: 20px;
}

#progressBar {
  height: 100%;
  width: 0%;
  background-color: #2563eb;
  border-radius: 4px;
  transition: width 0.3s ease;
}

ul {
  list-style: none;
  padding: 0;
  margin: 0;
}

li {
  display: flex;
  align-items: center;
  justify-content: space-between;
  background-color: #f9fafb;
  padding: 12px;
  margin-bottom: 10px;
  border: 1px solid #e5e7eb;
  border-radius: 6px;
  transition: background 0.2s ease;
}

li:hover {
  background-color: #f3f4f6;
}

.task-info {
  flex: 1;
  margin-left: 10px;
}
```

```
    font-size: 14px;
    color: #111827;
}

.task-info small {
    color: #6b7280;
    font-size: 12px;
}

.completed {
    text-decoration: line-through;
    color: #9ca3af;
}

.action-btn {
    border: none;
    padding: 6px 10px;
    margin-left: 6px;
    border-radius: 4px;
    font-size: 12px;
    cursor: pointer;
    transition: background 0.2s ease;
}

.edit {
    background-color: #e5e7eb;
    color: #111827;
}

.edit:hover {
    background-color: #d1d5db;
}

.delete {
    background-color: #ef4444;
    color: white;
}

.delete:hover {
    background-color: #dc2626;
}

.clear-btn {
```

```

width: 100%;
margin-top: 18px;
padding: 10px;
border: none;
border-radius: 6px;
background-color: #374151;
color: white;
font-size: 14px;
cursor: pointer;
transition: background 0.2s ease;
}

.clear-btn:hover {
  background-color: #1f2937;
}

@media (max-width: 480px) {
  .app-container {
    padding: 20px;
  }

  h1 {
    font-size: 20px;
  }
}

```

➤ JAVASCRIPT Code:

```

const taskInput = document.getElementById("taskInput");
const addTaskBtn = document.getElementById("addTaskBtn");
const taskList = document.getElementById("taskList");
const taskCount = document.getElementById("taskCount");
const completedCount = document.getElementById("completedCount");
const progressBar = document.getElementById("progressBar");
const clearAllBtn = document.getElementById("clearAllBtn");

let tasks = JSON.parse(localStorage.getItem("tasks")) || [];

function addTask(text) {
  const task = {
    id: Date.now(),
    text: text,
    completed: false,
    date: new Date().toLocaleString()
  };
}

```

```

    tasks.push(task);
    saveTasks();
    renderTasks();
}

function renderTasks() {
    taskList.innerHTML = "";
    let completedTasks = 0;

    tasks.forEach(task => {
        const li = document.createElement("li");

        const checkbox = document.createElement("input");
        checkbox.type = "checkbox";
        checkbox.checked = task.completed;

        const div = document.createElement("div");
        div.className = "task-info";
        div.innerHTML =
`<strong>${task.text}</strong><br><small>${task.date}</small>`;

        if (task.completed) {
            div.classList.add("completed");
            completedTasks++;
        }

        checkbox.addEventListener("change", () => {
            task.completed = checkbox.checked;
            saveTasks();
            renderTasks();
        });

        const editBtn = document.createElement("button");
        editBtn.textContent = "Edit";
        editBtn.className = "action-btn edit";
        editBtn.onclick = () => {
            const newText = prompt("Edit task:", task.text);
            if (newText && newText.trim() !== "") {
                task.text = newText.trim();
                saveTasks();
                renderTasks();
            }
        };

        const deleteBtn = document.createElement("button");
        deleteBtn.textContent = "Delete";
        deleteBtn.className = "action-btn delete";
    });
}

```

```

        deleteBtn.onclick = () => {
            tasks = tasks.filter(t => t.id !== task.id);
            saveTasks();
            renderTasks();
        };

        li.appendChild(checkbox);
        li.appendChild(div);
        li.appendChild(editBtn);
        li.appendChild(deleteBtn);

        taskList.appendChild(li);
    });

    updateStats(completedTasks);
}

function updateStats(completedTasks) {
    taskCount.textContent = `${tasks.length} Tasks`;
    completedCount.textContent = `${completedTasks} Completed`;

    const percent = tasks.length === 0 ? 0 :
        (completedTasks / tasks.length) * 100;
    progressBar.style.width = percent + "%";
}

function saveTasks() {
    localStorage.setItem("tasks", JSON.stringify(tasks));
}

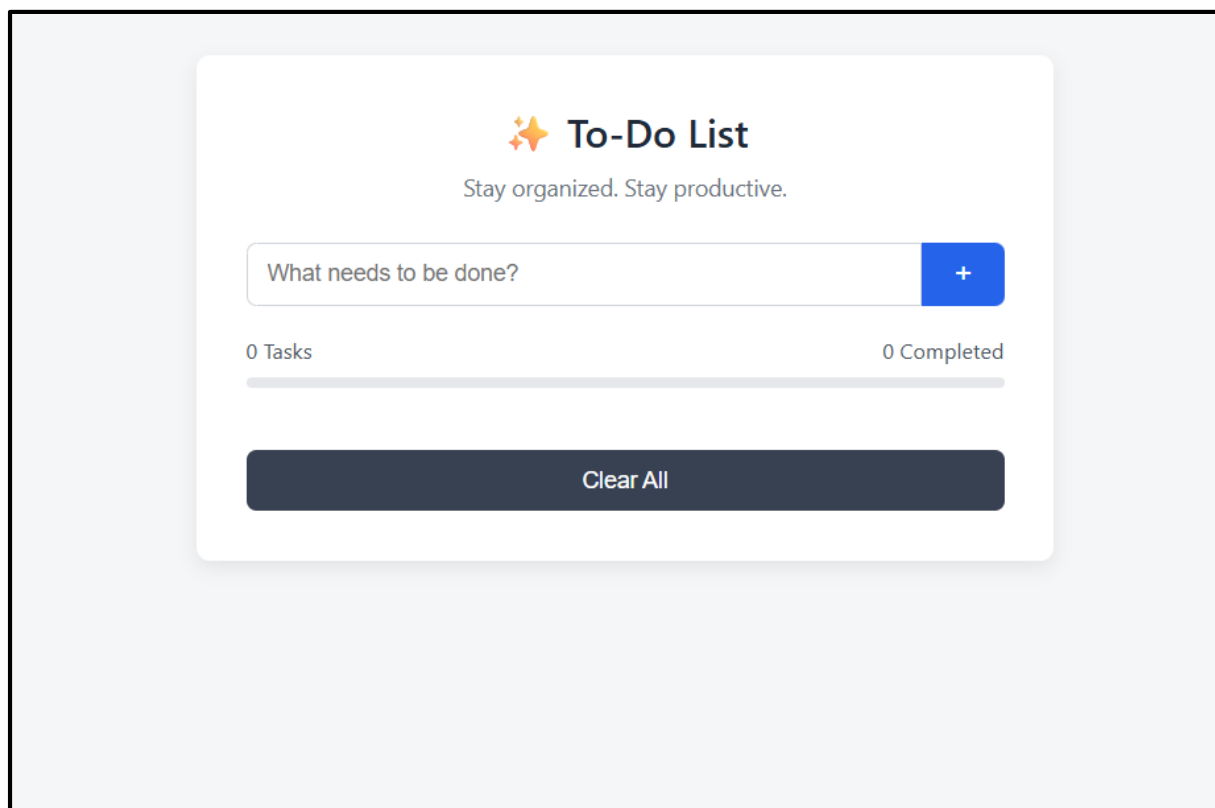
addTaskBtn.addEventListener("click", () => {
    const text = taskInput.value.trim();
    if (text !== "") {
        addTask(text);
        taskInput.value = "";
    }
});

taskInput.addEventListener("keypress", (e) => {
    if (e.key === "Enter") {
        addTaskBtn.click();
    }
});

```

```
clearAllBtn.addEventListener("click", () => {  
  if (confirm("Are you sure you want to delete all tasks?")) {  
    tasks = [];  
    saveTasks();  
    renderTasks();  
  }  
});  
  
renderTasks();
```

❖ OUTPUT-



The screenshot displays a web application titled "To-Do List" with a subtitle "Stay organized. Stay productive." The interface includes a text input field with the placeholder text "What needs to be done?" and a blue button with a white "+" symbol. Below the input field, it shows "0 Tasks" and "0 Completed" with a horizontal progress bar. At the bottom, there is a dark blue button labeled "Clear All".

ASSIGNMENT-5

❖ **AIM:** Design and develop a fully functional Event Management Website using HTML, CSS, JavaScript (front-end) and Python Flask (back-end), enabling users to create, browse, register for, and manage events through a dynamic and responsive web interface.

❖ **Tasks:**

- **Project Setup & Introduction** – Create a project folder named **event_management_website** containing the required files and folders: app.py, templates/, static/css/style.css, and static/js/script.js. Add header comments in all files (project title, student name, roll number, date). Configure the Flask application with debug mode and define the root route (/). Display a styled landing page with a brief description of the Event Management System.
- **Event Listing Page (HTML + Jinja2)** – Create a base template (base.html) with a navigation bar, header, and footer. Develop

an events.html page that extends the base template. Display a list or grid of events using semantic HTML elements like `<ul>` and `<li>`. Each event card should show the event name, date & time, venue, and a register button. Use Flask to pass event data (Python dictionaries) to the template dynamically.

- **Styling the Website (CSS)** – Apply a global CSS reset and define a consistent color scheme and typography. Style the navigation bar, hero section, event cards, and footer. Use Flexbox or Grid layout to create responsive designs. Add hover effects, transitions, and button animations. Ensure the website is mobile-responsive using media queries.
-

- **Event Registration Form (Flask + JavaScript Validation)** –

Create a registration page (register.html) with fields such as full name, email, phone number, event selection, and number of tickets. Handle form submission using a Flask POST route (/register). Implement client-side validation using JavaScript (check empty fields, email format, phone number). Display success or error messages after submission using Flask flash messages or alerts.

- **Admin Panel – Add / Edit / Delete Events** – Develop an admin page (/admin) accessible using session-based authentication. Create a form to add new events (name, date, venue, description, image URL). Display all events in a table with Edit and Delete options. Implement Flask routes for adding, editing, and deleting events (/admin/add, /admin/edit/, /admin/delete/). Use redirects and flash messages to confirm actions.

- **Search, Filter & RSVP Counter (Bonus)** – Add a live search bar on the events page using JavaScript to filter events without reloading the page. Implement category or date filters. Display RSVP counts for each event using Flask session or in-memory storage. Optionally integrate Flask-SQLAlchemy for database persistence.

❖ **Tech Stack Used:**

- **HTML, CSS, JavaScript, Python Flask and SQL**

❖ **CODES:**

➤ **PYTHON Code(app.py):**

```
from flask import Flask, render_template, request, redirect, flash, session  
app = Flask(__name__)
```

```

app.secret_key = "secret123"

# In-memory database
events = [
    {"id": 1, "name": "Music Concert", "date": "2026-05-10", "venue":
"Mumbai", "rsvp": 0},
    {"id": 2, "name": "Tech Conference", "date": "2026-06-15", "venue":
"Pune", "rsvp": 0},
    {"id": 3, "name": "Art Exhibition", "date": "2026-04-20", "venue":
"Delhi", "rsvp": 0},
    {"id": 4, "name": "Startup Meetup", "date": "2026-07-05", "venue":
"Bangalore", "rsvp": 0},
]

# ----- ROUTES ----- #

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/events")
def events_page():
    return render_template("events.html", events=events)

@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        event_name = request.form["event"]

        for event in events:
            if event["name"] == event_name:
                event["rsvp"] += 1

        flash("Registration Successful!", "success")
        return redirect("/events")

    return render_template("register.html", events=events)

# ----- ADMIN ----- #

@app.route("/admin")
def admin():
    if not session.get("admin"):
        session["admin"] = True # simple login bypass
    return render_template("admin.html", events=events)

@app.route("/admin/add", methods=["POST"])
def add_event():

```



```

new_event = {
    "id": len(events) + 1, "name":
    request.form["name"],

    "date": request.form["date"], "venue":
    request.form["venue"], "rsvp": 0

}

events.append(new_event) flash("Event
Added!", "success") return redirect("/admin")

@app.route("/admin/delete/<int:id>") def
delete_event(id):

    global events

    events = [e for e in events if e["id"] != id] flash("Event Deleted!", "danger")

    return redirect("/admin")
#
#

```

➤ HTML Code(admin.html):

```

{% extends "base.html" %}

{% block content %}
<h2>Admin Panel</h2>

<form method="POST" action="/admin/add">
<input type="text" name="name" placeholder="Event Name" required>
<input type="date" name="date" required>
<input type="text" name="venue" placeholder="Venue" required>
<button type="submit">Add Event</button>
</form>

<table>
<tr>
<th>Name</th>
<th>Date</th>
<th>Venue</th>
<th>Action</th>
</tr>

{% for event in events %}

```



```

<tr>
<td>{{ event.name }}</td>
<td>{{ event.date }}</td>
<td>{{ event.venue }}</td>
<td>
    <a href="/admin/delete/{{ event.id }}">Delete</a>
</td>
</tr>
{% endfor %}

</table>
{% endblock %}

```

➤ HTML Code(base.html):

```

<!DOCTYPE html>
<html>
<head>
    <title>EventHub</title>
    <link rel="stylesheet" href="{{ url_for('static',
filename='css/style.css') }}">
    <script defer src="{{ url_for('static', filename='js/script.js')
}}"></script>
</head>
<body>

<nav>
    <h2>EventHub</h2>
    <ul>
        <li><a href="/">Home</a></li>
        <li><a href="/events">Events</a></li>
        <li><a href="/register">Register</a></li>
        <li><a href="/admin">Admin</a></li>
    </ul>
</nav>

<div class="container">
    {% with messages = get_flashed_messages(with_categories=true) %}
        {% for category, message in messages %}
            <p class="{{ category }}">{{ message }}</p>
        {% endfor %}
    {% endwith %}

    {% block content %}{% endblock %}
</div>

<footer>

```



```
<p>© 2026 EventHub</p>
</footer>

</body>
</html>
```

➤ HTML Code(events.html):

```
{% extends "base.html" %}

{% block content %}
<h2>Events</h2>

<input type="text" id="search" placeholder="Search events..."
onkeyup="searchEvents()">

<div class="events" id="eventList">
  {% for event in events %}
    <div class="card">
      <h3>{{ event.name }}</h3>
      <p>{{ event.date }}</p>
      <p>{{ event.venue }}</p>
      <p>RSVP: {{ event.rsvp }}</p>
      <a href="/register" class="btn">Register</a>
    </div>
  {% endfor %}
</div>
{% endblock %}
```

➤ HTML Code(index.html):

```
{% extends "base.html" %}

{% block content %}
<div class="hero">
  <h1>Welcome to EventHub</h1>
  <p>Discover amazing events & register instantly!</p>
</div>
{% endblock %}
```

➤ HTML Code(register.html):

```
{% extends "base.html" %}
```



```
{% block content %}
<h2>Register</h2>

<form method="POST" onsubmit="return validateForm()">
  <input type="text" name="name" placeholder="Full Name" required>
  <input type="email" name="email" placeholder="Email" required>
  <input type="text" name="phone" placeholder="Phone Number" required>

  <select name="event">
    {% for event in events %}
      <option>{{ event.name }}</option>
    {% endfor %}
  </select>

  <input type="number" name="tickets" placeholder="Tickets" min="1">

  <button type="submit">Submit</button>
</form>
{% endblock %}
```

➤ CSS Code(style.css):

```
body {
  font-family: Arial;
  margin: 0;
  background: #f4f4f4;
}

nav {
  background: #222;
  color: white;
  display: flex;
  justify-content: space-between;
  padding: 15px;
}

nav ul {
  display: flex;
  gap: 20px;
  list-style: none;
}

nav a {
  color: white;
  text-decoration: none;
}
```



```

.hero {
  text-align: center;
  padding: 50px;
  background: linear-gradient(to right, #4facfe, #00f2fe);
  color: white;
}

.events {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
}

.card {
  background: white;
  padding: 20px;
  border-radius: 10px;
  transition: 0.3s;
}

.card:hover {
  transform: scale(1.05);
}

.btn {
  display: inline-block;
  background: #007bff;
  color: white;
  padding: 5px 10px;
}

.success {
  color: green;
}

.danger {

```

➤ JAVASCRIPT Code(script.js):

```

function validateForm() {
  let name = document.forms[0]["name"].value;
  let email = document.forms[0]["email"].value;
  let phone = document.forms[0]["phone"].value;

  if (name === "" || email === "" || phone === "") {

```



```

        alert("All fields required!");
        return false;
    }

    if (!/^\d{10}$/.test(phone)) {
        alert("Invalid phone number!");
        return false;
    }

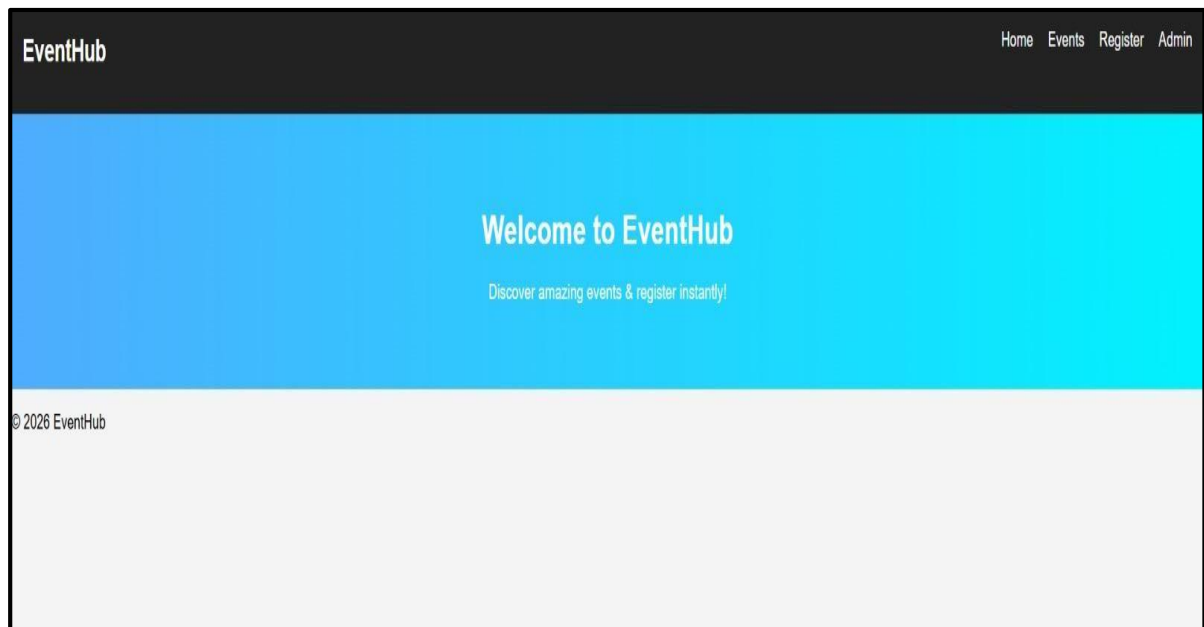
    return true;
}

function searchEvents() {
    let input = document.getElementById("search").value.toLowerCase();
    let cards = document.querySelectorAll(".card");

    cards.forEach(card => {
        let text = card.innerText.toLowerCase();
        card.style.display = text.includes(input) ? "block" : "none";
    });
}

```

❖ OUTPUT-



Events

Music Concert

2026-05-10

Mumbai

RSVP: 0

[Register](#)

Tech Conference

2026-06-15

Pune

RSVP: 0

[Register](#)

Art Exhibition

2026-04-20

Delhi

RSVP: 0

[Register](#)

Startup Meetup

2026-07-05

Bangalore

RSVP: 0

[Register](#)

© 2026 EventHub

ASSIGNMENT-6

➤ Tasks

Task 1: Project Setup & Environment

- Create a project folder named `contact_management_system`
- (Optional) Create and activate a virtual environment
- Install Flask using `pip install flask`
- Create the following structure:
 - o `app.py`
 - o `templates/`
 - ? `base.html`
 - ? `index.html`
 - ? `add_contact.html`
 - ? `edit_contact.html`
 - o `static/` (optional for CSS)
- Add header comments in `app.py` (project title, name, date)

Deliverable:

Flask project structure with required files.

Task 2: Flask App Initialization

- Import Flask and required modules
- Initialize the Flask application
- Create a home route (`/`) to display all contacts
- Use a list or dictionary to store contact data (no database required)

Deliverable:

Running Flask app showing an empty or sample contact list.

Task 3: Add New Contacts (Create Operation)

- Create a route and form to add new contacts
- Accept at least:
 - o Name
 - o Phone number
 - o Email address
- Validate inputs to avoid empty fields

Deliverable:

Functionality to add and display new contacts.

Task 4: View & Display Contacts (Read Operation)

- Display all stored contacts in a table or list
- Show contact details clearly
- Add navigation links for better usability

Deliverable:

Readable contact list using HTML templates.

Task 5: Update Contact Details (Update Operation)

- Create an edit route for existing contacts
- Pre-fill form fields with current contact details
- Save updated information correctly

Deliverable:

Working contact update functionality.

Task 6: Delete Contacts (Delete Operation)

- Add a delete option for each contact
- Remove selected contact from storage

- Redirect to home page after deletion

Deliverable:

Delete functionality implemented correctly.

Task 7 (Bonus): Search & Styling

- Add a simple search feature by name or phone number
- Apply basic CSS styling for better UI
- Improve layout using tables or cards

Deliverable:

Enhanced contact management system with search and styling.

➤ **Submission Instructions**

- contact_management_system project folder
- app.py
- templates directory with HTML files
- static folder (if used)
- At least 2 screenshots (contact list & add/edit contact)
- Clean, commented code
- GitHub repository URL submission via LMS.

➤ **CODES:**

- **PYTHON Code(app.py)**

```
from flask import Flask, render_template, request, redirect, url_for

app = Flask(__name__)

# In-memory storage
contacts = []
contact_id_counter = 1

@app.route('/')
```

```

def index():
    search_query = request.args.get('search', '')
    if search_query:
        filtered = [c for c in contacts if search_query.lower() in c['name'].lower() or
search_query in c['phone']]
    else:
        filtered = contacts
    return render_template('index.html', contacts=filtered)

@app.route('/add', methods=['GET', 'POST'])
def add_contact():
    global contact_id_counter
    if request.method == 'POST':
        name = request.form['name']
        phone = request.form['phone']
        email = request.form['email']

        if not name or not phone or not email:
            return "All fields required!"

        contacts.append({
            'id': contact_id_counter,
            'name': name,
            'phone': phone,
            'email': email
        })
        contact_id_counter += 1
        return redirect(url_for('index'))

    return render_template('add_contact.html')

@app.route('/edit/<int:id>', methods=['GET', 'POST'])
def edit_contact(id):
    contact = next((c for c in contacts if c['id'] == id), None)

    if request.method == 'POST':
        contact['name'] = request.form['name']
        contact['phone'] = request.form['phone']
        contact['email'] = request.form['email']
        return redirect(url_for('index'))

    return render_template('edit_contact.html', contact=contact)

@app.route('/delete/<int:id>')
def delete_contact(id):
    global contacts
    contacts = [c for c in contacts if c['id'] != id]
    return redirect(url_for('index'))

```

```
if __name__ == '__main__':
    app.run(debug=True)
```

- **HTML Code(index.html)**

```
{% extends 'base.html' %}

{% block content %}
<h2>Contact List</h2>

<form method="GET">
    <input type="text" name="search" placeholder="Search...">
    <button type="submit">Search</button>
</form>

<table>
    <tr>
        <th>Name</th>
        <th>Phone</th>
        <th>Email</th>
        <th>Actions</th>
    </tr>

    {% for contact in contacts %}
    <tr>
        <td>{{ contact.name }}</td>
        <td>{{ contact.phone }}</td>
        <td>{{ contact.email }}</td>
        <td>
            <a href="/edit/{{ contact.id }}">Edit</a>
            <a href="/delete/{{ contact.id }}">Delete</a>
        </td>
    </tr>
    {% endfor %}
</table>
{% endblock %}
```

- **HTML Code(base.html)**

```
<!DOCTYPE html>
<html>
<head>
    <title>Contact Manager</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
    <nav>
        <h2>📞 Contact Manager</h2>
        <a href="/">Home</a>
        <a href="/add">Add Contact</a>
    </nav>

    <div class="container">
```

```

        {% block content %}{% endblock %}
    </div>
</body>
</html>

```

- **HTML Code(add_contact.html)**

```

{% extends 'base.html' %}

{% block content %}
<h2>Add Contact</h2>

<form method="POST">
    <input type="text" name="name" placeholder="Name" required>
    <input type="text" name="phone" placeholder="Phone" required>
    <input type="email" name="email" placeholder="Email" required>
    <button type="submit">Add</button>
</form>
{% endblock %}

```

- **HTML Code(edit_contact.html)**

```

{% extends 'base.html' %}

{% block content %}
<h2>Edit Contact</h2>

<form method="POST">
    <input type="text" name="name" value="{{ contact.name }}" required>
    <input type="text" name="phone" value="{{ contact.phone }}" required>
    <input type="email" name="email" value="{{ contact.email }}" required>
    <button type="submit">Update</button>
</form>
{% endblock %}

```

- **CSS Code(style.css)**

```

body {
    font-family: Arial;
    background: #f4f6f9;
}

nav {
    background: #333;
    color: white;
    padding: 10px;
}

nav a {
    color: white;
    margin-left: 15px;
}

.container {

```

```
padding: 20px;
}

table {
  width: 100%;
  background: white;
  border-collapse: collapse;
}

table, th, td {
  border: 1px solid #ddd;
  padding: 10px;
}

button {
  background: #007bff;
  color: white;
  padding: 8px;
  border: none;
}
```

➤ OUTPUT:

The screenshot displays a CRM application interface. On the left is a sidebar with a 'Workspace' section containing a 'CRM' dropdown and buttons for '+ Add', 'Filters', and 'Search'. Below these are navigation links for 'Deals', 'Contacts' (highlighted), 'Accounts', 'Leads', 'Activities', 'Getting Started', and 'Sales Dashboard'. The main area is titled 'Contacts' and features a 'Main Table' tab. A 'New Contact' button is visible. The table has columns for 'Contact', 'Title', 'Priority', 'Phone', and 'Email'. It lists three contacts: Phoenix Levy (COO, High priority), Madison Doyle (CEO, High priority), and Leilani Krause (CIO, Medium priority). At the bottom of the table is an 'Add new group' button.

Contact	Title	Priority	Phone	Email
Phoenix Levy	COO	High	+1 325 478 5698	Phoenix@email.com
Madison Doyle	CEO	High	+1 854 722 0499	Madison@email.com
Leilani Krause	CIO	Medium	+1 312 563 2541	Leilani@email.com

ASSIGNMENT-7

➤ Tasks:

Task 1: Project Setup & Structure

- Create a project folder named ecommerce_website
- Inside the folder, create:
 - o index.html
 - o style.css
 - o script.js
- Add header comments in HTML (project title, name, date)
- Create a homepage introducing the e-commerce website

Deliverable:

Project folder with HTML, CSS, and JS files linked correctly.

Task 2: Product Listing Page

- Display a list of products with:
 - o Product name
 - o Price
 - o Short description
- Use cards or grid layout for product display
- Add an “Add to Cart” button for each product

Deliverable:

Product listing section with at least 3 products.

Task 3: Styling & Layout (CSS)

- Apply CSS for:
 - o Product cards

- o Buttons and typography
- o Page layout and spacing
- Ensure consistent styling across pages/sections

Deliverable:

Visually neat and user-friendly style.css.

Task 4: Shopping Cart Functionality (JavaScript)

- Add selected products to a shopping cart
- Display cart items dynamically
- Show item quantity and total price
- Allow users to remove items from the cart

Deliverable:

Working shopping cart functionality using JavaScript.

Task 5: Cart Summary & Price Calculation

- Calculate:
 - o Individual item totals
 - o Overall cart total
- Update totals dynamically when items are added or removed

Deliverable:

Accurate and dynamic cart price calculations.

Task 6: Checkout Simulation

- Create a checkout section or page
- Collect basic user details (Name, Address – optional)
- Display a purchase confirmation message (no real payment gateway)

Deliverable:

Simulated checkout and order confirmation.

Task 7 (Bonus): Enhanced Features

- Add quantity update buttons (+ / –)
- Store cart data temporarily using browser storage (optional)
- Improve UI with hover effects or transitions

Deliverable:

Enhanced e-commerce experience with additional features.

Submission Instructions

- ecommerce_website project folder
- index.html, style.css, script.js
- At least 3 screenshots (products, cart, checkout)
- Clean, well-commented code
- GitHub repository URL submission via LMS.

➤ CODES:

- HTML Code(index.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>E-Commerce Website</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
  <h1>🛒 MyStore</h1>
</header>

<div class="container">

  <!-- PRODUCTS -->
  <h2>Products</h2>
  <div id="products" class="product-grid"></div>
```

```

<!-- CART -->
<div class="cart-section">
  <h2>🛒 Cart</h2>
  <div id="cart"></div>
  <h3>Total: ₹<span id="total">0</span></h3>
</div>

<!-- CHECKOUT -->
<div class="checkout">
  <h2>Checkout</h2>
  <input type="text" id="name" placeholder="Your Name">
  <input type="text" id="address" placeholder="Address">
  <button onclick="checkout()">Place Order</button>
</div>

</div>

<script src="script.js"></script>
</body>
</html>

```

- **CSS Code(style.css)**

```

body {
  font-family: Arial, sans-serif;
  margin: 0;
  background: #f5f7fa;
}

header {
  background: #222;
  color: white;
  padding: 15px;
  text-align: center;
}

.container {
  padding: 20px;
}

/* PRODUCT GRID */
.product-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}

/* CARD */
.card {
  background: white;
  padding: 15px;
  border-radius: 12px;
}

```

```

    box-shadow: 0 4px 10px rgba(0,0,0,0.1);
    text-align: center;
    transition: 0.3s;
}

.card:hover {
    transform: translateY(-5px);
}

/* IMAGE */
.card img {
    width: 100%;
    height: 150px;
    object-fit: cover;
    border-radius: 10px;
}

/* BUTTON */
button {
    background: #28a745;
    color: white;
    border: none;
    padding: 8px 12px;
    margin-top: 5px;
    border-radius: 5px;
    cursor: pointer;
}

button:hover {
    background: #218838;
}

/* CART */
.cart-section {
    margin-top: 30px;
    background: white;
    padding: 15px;
    border-radius: 10px;
}

/* CHECKOUT */
.checkout {
    margin-top: 30px;
    background: white;
    padding: 15px;
    border-radius: 10px;
}

input {
    width: 100%;
    padding: 10px;
    margin: 5px 0;
}

```

```
}
```

- JavaScript Code(script.js)

```
const products = [
  {id: 1, name: "Laptop", price: 50000},
  {id: 2, name: "Phone", price: 20000},
  {id: 3, name: "Headphones", price: 2000}
];

let cart = [];

function renderProducts() {
  let html = "";
  products.forEach(p => {
    html += `
    <div class="card">
      <h3>${p.name}</h3>
      <p>₹${p.price}</p>
      <button onclick="addToCart(${p.id})">Add to Cart</button>
    </div>`;
  });
  document.getElementById("products").innerHTML = html;
}

function addToCart(id) {
  const product = products.find(p => p.id === id);
  cart.push(product);
  renderCart();
}

function renderCart() {
  let html = "";
  let total = 0;

  cart.forEach((item, index) => {
    total += item.price;
    html += `
    <p>${item.name} - ₹${item.price}</p>
    <button onclick="removeItem(${index})">Remove</button></p>`;
  });

  document.getElementById("cart").innerHTML = html;
  document.getElementById("total").innerText = total;
}

function removeItem(index) {
  cart.splice(index, 1);
  renderCart();
}

function checkout() {
```

```

let name = document.getElementById("name").value;
if(name === "") {
    alert("Enter name");
    return;
}
alert("Order placed successfully!");
cart = [];
renderCart();
}

renderProducts();

```

➤ OUTPUT:

 **MyStore**

Products

Laptop
₹50000
Add to Cart

Phone
₹20000
Add to Cart

Headphones
₹2000
Add to Cart

 **Cart**
Total: ₹0

Checkout

Products

Laptop
₹50000
Add to Cart

Phone
₹20000
Add to Cart

Headphones
₹2000
Add to Cart

 **Cart**
Total: ₹0

Checkout

